PS3

April 6, 2026

1 Problem Set #3 — Computing the Aiyagari Economy

Nima Nikopour

Due: April 6th, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.interpolate import interp1d

# Model parameters
beta = 0.96         # discount factor
sigma = 2.0         # CRRA coefficient
alpha = 1/3         # capital share
delta = 0.05        # depreciation rate
A = 1.0             # TFP
b = 0.0             # borrowing limit parameter

# Skill levels
s_L = 0.25
s_H = 1.0
s_vals = np.array([s_L, s_H])

# Transition matrix (rows = current state, cols = next state)
# Pi[i, j] = Pr(s' = s_j | s = s_i)
Pi = np.array([
    [0.75, 0.25],   # from s_L: stay low 0.75, go high 0.25
    [0.15, 0.85]    # from s_H: go low 0.15, stay high 0.85
])
```

1.0.1 Question 1

Let's start by characterizing the skill (and with it the income) distribution.

Q1(a) Write down the transition probability matrix of s, $\pi(s'|s)$ carefully defining rows or columns (up to you, as long as you are consistent) as the respective. Is there an invariant distribution $\lambda(s)$? Is it unique? Explain.

Answer:

We define the transition probability matrix Π with rows as the current state and columns as the next state, i.e., $\Pi_{ij} = \pi(s_j|s_i)$:

$$\Pi = \begin{pmatrix} \pi(s_L|s_L) & \pi(s_H|s_L) \\ \pi(s_L|s_H) & \pi(s_H|s_H) \end{pmatrix} = \begin{pmatrix} 0.75 & 0.25 \\ 0.15 & 0.85 \end{pmatrix}$$

Existence and uniqueness of the invariant distribution: An invariant (stationary) distribution $\lambda = (\lambda(s_L), \lambda(s_H))$ satisfies $\lambda'\Pi = \lambda'$ and $\lambda(s_L) + \lambda(s_H) = 1$. Since all entries of Π are strictly positive, the Markov chain is irreducible (every state can be reached from every other state) and aperiodic (the positive diagonal entries ensure the chain does not cycle). By the Perron-Frobenius theorem, there exists a unique stationary distribution.

Solving analytically from $\lambda(s_L) \cdot 0.75 + \lambda(s_H) \cdot 0.15 = \lambda(s_L)$:

$$0.25\,\lambda(s_L) = 0.15\,\lambda(s_H) \implies \lambda(s_H) = \frac{5}{3}\,\lambda(s_L)$$

With the normalization $\lambda(s_L) + \lambda(s_H) = 1$, we get $\lambda(s_L) = \frac{3}{8} = 0.375$ and $\lambda(s_H) = \frac{5}{8} = 0.625$.

[2]:
```
# Q1(a): Compute the invariant distribution
# Solve lambda' @ Pi = lambda' subject to sum(lambda) = 1
# Equivalently, find the left eigenvector with eigenvalue 1

# Method: solve (Pi' - I) @ lambda = 0 with normalization
A_mat = Pi.T - np.eye(2)
A_mat[-1, :] = 1.0          # replace last equation with normalization
b_vec = np.array([0.0, 1.0])
lam = np.linalg.solve(A_mat, b_vec)

print(f"Invariant distribution:  (s_L) = {lam[0]:.4f},  (s_H) = {lam[1]:.4f}")
print(f"Check: {lam[0]:.4f} + {lam[1]:.4f} = {lam.sum():.4f}")
print(f"Verification  'Π =  ': {np.allclose(lam @ Pi, lam)}")
```

```
Invariant distribution:  (s_L) = 0.3750,  (s_H) = 0.6250
Check: 0.3750 + 0.6250 = 1.0000
Verification  'Π =  ': True
```

Q1(b) Compute the mean, standard deviation, and the coefficient of autocorrelation (t with $t-1$) of s and $\ln s$.

[3]:
```
# Q1(b): Mean, std, autocorrelation of s and ln(s)

# --- Statistics for s ---
# Mean: E[s] = sum of s * lambda(s)
mean_s = np.dot(s_vals, lam)

# Variance: E[s^2] - (E[s])^2
```

```python
mean_s2 = np.dot(s_vals**2, lam)
var_s = mean_s2 - mean_s**2
std_s = np.sqrt(var_s)

# Autocorrelation: corr(s_t, s_{t-1})
# E[s_t * s_{t-1}] = sum over (s, s') of s * s' * pi(s'|s) * lambda(s)
E_s_sprime = 0.0
for i in range(2):
    for j in range(2):
        E_s_sprime += s_vals[i] * s_vals[j] * Pi[i, j] * lam[i]
cov_s = E_s_sprime - mean_s**2
autocorr_s = cov_s / var_s

print("=== Statistics for s ===")
print(f"Mean(s)     = {mean_s:.4f}")
print(f"Std(s)      = {std_s:.4f}")
print(f"Autocorr(s) = {autocorr_s:.4f}")

# --- Statistics for ln(s) ---
ln_s_vals = np.log(s_vals)

mean_ln_s = np.dot(ln_s_vals, lam)
mean_ln_s2 = np.dot(ln_s_vals**2, lam)
var_ln_s = mean_ln_s2 - mean_ln_s**2
std_ln_s = np.sqrt(var_ln_s)

E_ln_s_sprime = 0.0
for i in range(2):
    for j in range(2):
        E_ln_s_sprime += ln_s_vals[i] * ln_s_vals[j] * Pi[i, j] * lam[i]
cov_ln_s = E_ln_s_sprime - mean_ln_s**2
autocorr_ln_s = cov_ln_s / var_ln_s

print("\n=== Statistics for ln(s) ===")
print(f"Mean(ln s)     = {mean_ln_s:.4f}")
print(f"Std(ln s)      = {std_ln_s:.4f}")
print(f"Autocorr(ln s) = {autocorr_ln_s:.4f}")
```

```
=== Statistics for s ===
Mean(s)     = 0.7188
Std(s)      = 0.3631
Autocorr(s) = 0.6000

=== Statistics for ln(s) ===
Mean(ln s)     = -0.5199
Std(ln s)      = 0.6711
Autocorr(ln s) = 0.6000
```

1.0.2 Question 2

Let's now characterize the prices of labor and capital as implied by q. To this end, let

$$\bar{L} = \sum_{s \in \{s_L, s_H\}} s\,\lambda(s),$$

where $\lambda(\cdot)$ is (one of?) the invariant distributions of labor market skills that you computed above.

Q2(a) Given $L = \bar{L}$ and for any K, write down the competitive factor prices w, r, as defined by the profit maximization of firms in the economy.

Answer:

Firms maximize profits by choosing K and L:

$$\max_{K,L} \; AK^{\alpha}L^{1-\alpha} - rK - wL$$

Under perfect competition, factors are paid their marginal products. Taking derivatives:

$$r = \alpha A K^{\alpha-1} L^{1-\alpha}$$

$$w = (1-\alpha) A K^{\alpha} L^{-\alpha}$$

where r is the rental rate of capital and w is the wage per unit of skill. Aggregate labor is fixed at $L = \bar{L} = \sum_s s\,\lambda(s)$.

Q2(b) Given your answers above, write down the implied capital $K(q)$, and the implied wages associated to $w(q)$. Why is it that there is a unique $w(q)$ associated to each q?

Answer:

The bond price q is related to the interest rate by $R = 1/q$, so the net interest rate is $1/q - 1$. No-arbitrage between bonds and capital requires the net return on capital to equal the net return on bonds:

$$\frac{1}{q} - 1 = r - \delta \quad \Longrightarrow \quad r = \frac{1}{q} - 1 + \delta$$

From Q2(a), we have $r = \alpha A K^{\alpha-1} \bar{L}^{1-\alpha}$. Setting these equal and solving for K:

$$K(q) = \left(\frac{\alpha A}{\frac{1}{q} - 1 + \delta} \right)^{\frac{1}{1-\alpha}} \bar{L}$$

Substituting $K(q)$ into the wage expression from Q2(a):

$$w(q) = (1-\alpha)A\left(\frac{K(q)}{\bar{L}}\right)^{\alpha} = (1-\alpha)A\left(\frac{\alpha A}{\frac{1}{q}-1+\delta}\right)^{\frac{\alpha}{1-\alpha}}$$

Why is $w(q)$ unique for each q? Each step is a one-to-one mapping: q uniquely determines r (via no-arbitrage), r uniquely determines K (since $r = MPK$ is strictly decreasing in K, so the inverse is unique), and K uniquely determines w (via the wage formula). So each q maps to exactly one w.

[4]:
```
# Q2: Compute L_bar and define functions for K(q), w(q), r(q)
L_bar = np.dot(s_vals, lam)
print(f"L_bar = {L_bar:.4f}")

def r_from_q(q):
    return 1/q - 1 + delta

def K_from_q(q):
    r = r_from_q(q)
    return ((alpha * A / r) ** (1/(1-alpha))) * L_bar

def w_from_q(q):
    K = K_from_q(q)
    return (1-alpha) * A * (K / L_bar)**alpha

# Quick check
q_test = 0.98
print(f"\nAt q = {q_test}:")
print(f"  r = {r_from_q(q_test):.4f}")
print(f"  K = {K_from_q(q_test):.4f}")
print(f"  w = {w_from_q(q_test):.4f}")
```

```
L_bar = 0.7188

At q = 0.98:
  r = 0.0704
  K = 7.4039
  w = 1.4506
```

1.0.3 Question 3

Finally, we proceed to solve for the stationary GE of the model. Here, we will do it using a rudimentary, bare-bones strategy. In practice, you can use a more iterative algorithm. Anyway, define an equally spaced grid for q of $N_q = 100$ points in the interval $[1.02 \times \beta,\ 0.999]$. Why above β?

Answer:

We need $q > \beta$ for a stationary distribution to exist. Consider the household Euler equation:

$$u'(c_t) = \beta R \cdot E_t[u'(c_{t+1})]$$

where $R = 1/q$ is the gross interest rate. If $q = \beta$, then $\beta R = 1$ and the Euler equation becomes $u'(c_t) = E_t[u'(c_{t+1})]$. Since u' is convex ($\sigma = 2$), Jensen's inequality implies $E[u'(c_{t+1})] > u'(E[c_{t+1}])$, so $c_t < E[c_{t+1}]$ — consumption drifts upward over time and households accumulate wealth indefinitely. No stationary distribution exists.

When $q > \beta$ (i.e., $\beta R < 1$), discounting dominates and households' desire to save is bounded, allowing wealth to stabilize at a stationary distribution.

The lower bound $1.02 \times \beta$ gives a small buffer above β. The upper bound $0.999 < 1$ ensures $R = 1/q > 1$, i.e., a positive net interest rate, which is needed for $K(q) > 0$.

Q3(a)(i) Write down the BE of the worker, explaining carefully what are the "state" for the workers' problem given that q is constant. Can you argue that there exists a unique solution? Would you get it if you start your iteration from $V = 0$?

Answer:

Given a fixed bond price q, the worker's state is (a, s) — current asset holdings a and current skill level $s \in \{s_L, s_H\}$. The only decision is how much to save, i.e., choosing next-period assets a'. The Bellman equation is:

$$V(a, s) = \max_{a' \geq \phi} \left\{ u\left(ws + a - qa'\right) + \beta \sum_{s'} \pi(s'|s)\, V(a', s') \right\}$$

subject to $c = ws + a - qa' \geq 0$, where $w = w(q)$ and ϕ is the borrowing limit. With $b = 0$, the natural borrowing limit is $\phi = \min\left\{0,\ \frac{ws_L}{1-q}\right\} = 0$ (since the natural limit $\frac{ws_L}{1-q} > 0$). So households simply cannot borrow: $a' \geq 0$.

Unique solution: The Bellman operator T defined by the right-hand side is a contraction mapping on the space of bounded continuous functions (Blackwell's sufficient conditions hold: monotonicity and discounting with $\beta < 1$ and $q > \beta$). By the Contraction Mapping Theorem, there exists a unique fixed point V.

Starting from $V = 0$: Yes. Since T is a contraction, repeated application $T^n(V_0) \to V^*$ for any initial guess V_0, including $V_0 = 0$. This is exactly what value function iteration does.

Q3(a)(ii) Write down an algorithm to numerically solve for the value and policy function (V, g) for the worker. Here, you may want to define a grid of points for wealth $\mathcal{A}$, and either use straight discretization or Chebyshev polynomials or splines or other methods to solve the problem. In either case, make sure that your computations do not depend on upper bound and the coarseness (the number of points) of the grid for $\mathcal{A}$. For example, make sure that changes in those do not affect the results, nor that there are a significant fraction of agents in the upper bound. Please, graph for each separate s, the value function V (Figure 1) and policy function g (Figure 2) with a in the horizontal axis. Please, add your code with your answers.

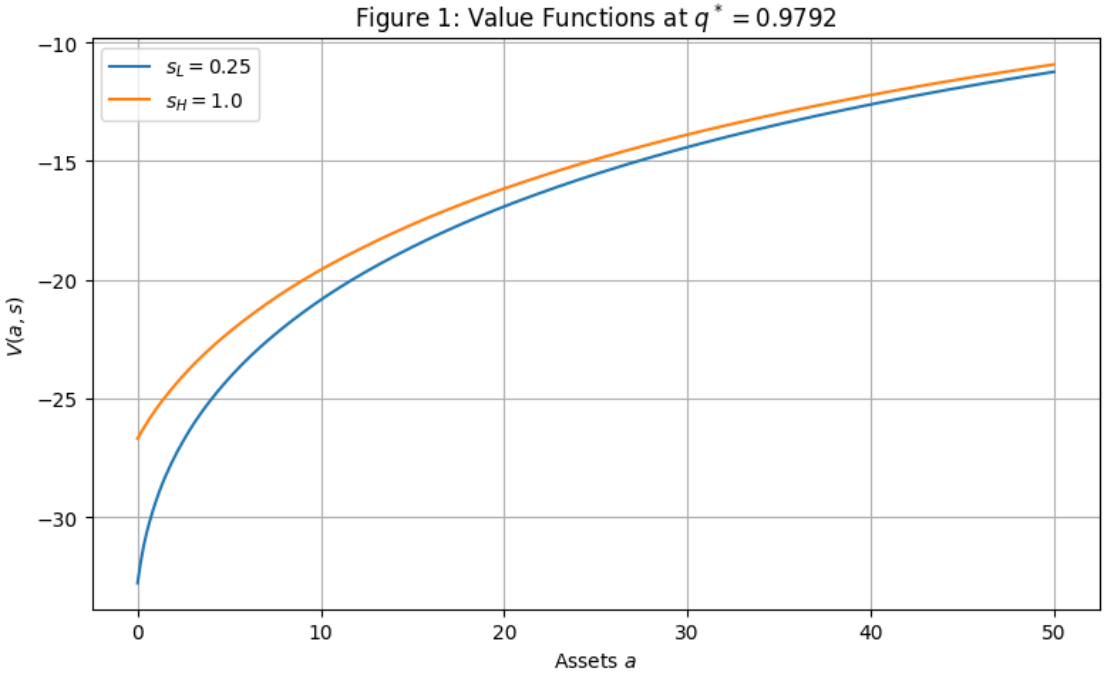
[29]:
```
# Q3(a)(ii): Grids, VFI solver, stationary distribution, and main computation

def u_crra(c):
    """CRRA utility with sigma=2: u(c) = c^(1-sigma)/(1-sigma)"""
    return c**(1 - sigma) / (1 - sigma)

# --- Grid setup ---
N_a = 300                # asset grid points
a_max = 50.0             # upper bound on assets (will verify this is large␣
 ↪enough)
a_grid = np.linspace(0, a_max, N_a)

N_q = 100
q_grid = np.linspace(1.02 * beta, 0.999, N_q)

def solve_vfi(q, tol=1e-6, max_iter=2000):
    """Solve the household problem for a given bond price q via value function␣
 ↪iteration."""
    w = w_from_q(q)

    # Precompute utility for every (current asset i, next asset k) pair, for␣
 ↪each skill j
    # c[i, k] = w*s_j + a_grid[i] - q*a_grid[k]
    u_mat = []
    for j in range(2):
        c = (w * s_vals[j] + a_grid)[:, None] - q * a_grid[None, :]
        uj = np.full_like(c, -np.inf)
        pos = c > 0
        uj[pos] = u_crra(c[pos])
        u_mat.append(uj)

    # Value function iteration starting from V = 0
    V = np.zeros((N_a, 2))
    g_idx = np.zeros((N_a, 2), dtype=int)

    for it in range(max_iter):
        V_new = np.zeros_like(V)
        for j in range(2):
            EV = V @ Pi[j]                                  # (N_a,) expected␣
 ↪value at each a'
            val = u_mat[j] + beta * EV[np.newaxis, :]      # (N_a, N_a)
            g_idx[:, j] = np.argmax(val, axis=1)
            V_new[:, j] = np.max(val, axis=1)
        if np.max(np.abs(V_new - V)) < tol:
            break
        V = V_new.copy()
```

```
    return V, a_grid[g_idx], g_idx

def compute_stationary_dist(g_idx, tol=1e-10, max_iter=10000):
    """Compute stationary distribution over (asset, skill) given policy␣
 ↪function indices."""
    dist = np.ones((N_a, 2)) / (N_a * 2)
    for it in range(max_iter):
        dist_new = np.zeros_like(dist)
        for j in range(2):           # current skill
            for jp in range(2):      # next skill
                np.add.at(dist_new[:, jp], g_idx[:, j], Pi[j, jp] * dist[:, j])
        if np.max(np.abs(dist_new - dist)) < tol:
            break
        dist = dist_new
    return dist

# === Main computation loop: solve for all q values ===
print("Solving household problem for each q value...")
V_all = []
g_all = []
g_idx_all = []
dist_all = []
Ea_all = np.zeros(N_q)
K_demand = np.zeros(N_q)

for iq, q_val in enumerate(q_grid):
    V, g, g_idx = solve_vfi(q_val)
    V_all.append(V)
    g_all.append(g)
    g_idx_all.append(g_idx)

    dist = compute_stationary_dist(g_idx)
    dist_all.append(dist)
    Ea_all[iq] = np.sum(a_grid[:, None] * dist)
    K_demand[iq] = K_from_q(q_val)

    if (iq + 1) % 20 == 0:
        print(f"  [{iq+1:3d}/{N_q}] q={q_val:.4f}  K(q)={K_demand[iq]:7.2f} ␣
 ↪Ea(q)={Ea_all[iq]:7.2f}")

# --- Find approximate equilibrium: where K(q)  Ea(q) ---
eq_idx = np.argmin(np.abs(K_demand - Ea_all))
q_star = q_grid[eq_idx]

print(f"\nApproximate equilibrium: q*  {q_star:.4f}")
print(f"  K(q*)  = {K_demand[eq_idx]:.4f}")
print(f"  Ea(q*) = {Ea_all[eq_idx]:.4f}")
```

```
print(f"  |K - Ea| = {abs(K_demand[eq_idx] - Ea_all[eq_idx]):.4f}")

# Verify upper bound is adequate
frac_at_top = np.sum(dist_all[eq_idx][-1, :])
print(f"  Fraction of agents at a_max = {frac_at_top:.6f} (should be  0)")
```

```
Solving household problem for each q value…
  [ 20/100] q=0.9830  K(q)=   7.92  Ea(q)=   4.19
  [ 40/100] q=0.9870  K(q)=   8.71  Ea(q)=   3.84
  [ 60/100] q=0.9910  K(q)=   9.63  Ea(q)=   3.62
  [ 80/100] q=0.9950  K(q)=  10.72  Ea(q)=   3.40
  [100/100] q=0.9990  K(q)=  12.01  Ea(q)=   3.27

Approximate equilibrium: q*  0.9792
  K(q*)  = 7.2743
  Ea(q*) = 4.5667
  |K - Ea| = 2.7077
  Fraction of agents at a_max = 0.000000 (should be  0)
```

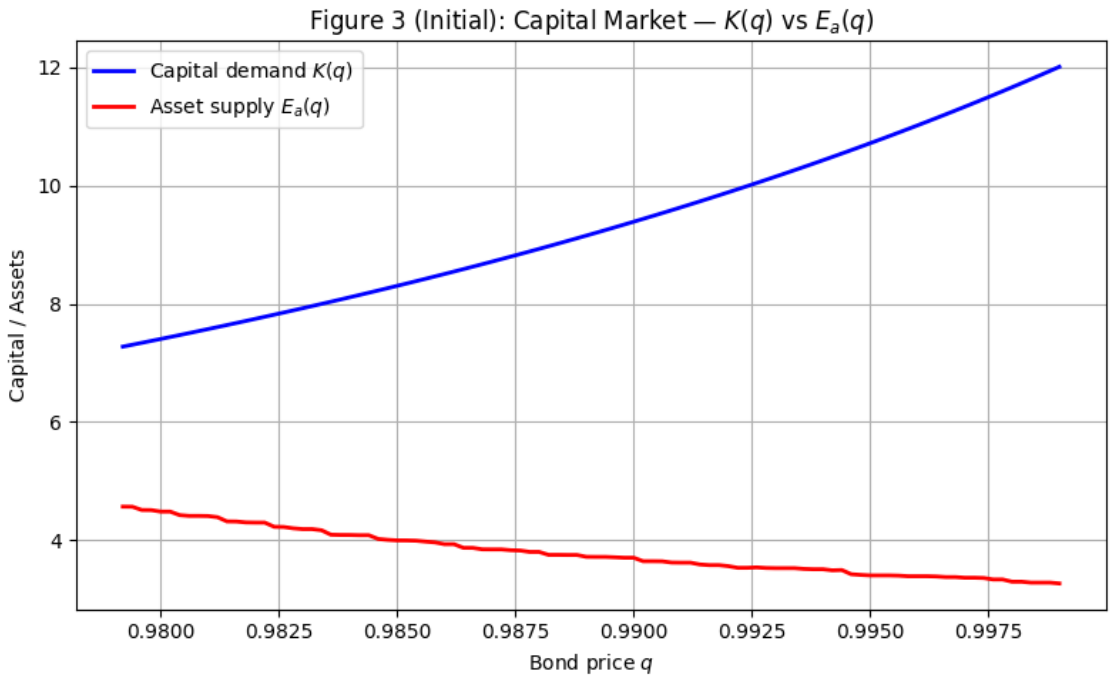
[ ]:
```
*Note: Figures 1 and 2 are presented below, after the equilibrium $q^*$ has␣
↪been identified and corrected in Q3(a)(iii), so they display the value and␣
↪policy functions at the true equilibrium.*
```



Q3(a)(iii) Given the policy function g, characterize the invariant distribution of wealth $a \in \mathcal{A}$. Compute the level of aggregate wealth, denoted $E_a(q)$ associate to each q. Produce the graph with

$K(q)$ and $E_a(q)$ for all the q in the grid. Indicate which one(s) correspond to the (approximate) stationary equilibrium if any. If none, discuss how you amended your code to sort it out (e.g. finer grids, an alternative iterative algorithm, etc. Yes, that is the right attitude!!)

[34]:
```
# Q3(a)(iii): Initial attempt - Graph K(q) vs E_a(q)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(q_grid, K_demand, 'b-', linewidth=2, label=r'Capital demand $K(q)$')
ax.plot(q_grid, Ea_all, 'r-', linewidth=2, label=r'Asset supply $E_a(q)$')
ax.set_xlabel('Bond price $q$')
ax.set_ylabel('Capital / Assets')
ax.set_title('Figure 3 (Initial): Capital Market - $K(q)$ vs $E_a(q)$')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()

# Check: do the curves cross?
print(f"K(q) range: [{K_demand.min():.2f}, {K_demand.max():.2f}]")
print(f"Ea(q) range: [{Ea_all.min():.2f}, {Ea_all.max():.2f}]")
gap = K_demand - Ea_all
print(f"K(q) - Ea(q) is always positive: {np.all(gap > 0)}")
print(f"Smallest gap = {gap.min():.4f} at q = {q_grid[np.argmin(gap)]:.4f}")
print("\n=> The curves do NOT cross on this grid. No equilibrium found!")
```




```
K(q) range: [7.27, 12.01]
```

```
Ea(q) range: [3.27, 4.57]
K(q) - Ea(q) is always positive: True
Smallest gap = 2.7077 at q = 0.9792

=> The curves do NOT cross on this grid. No equilibrium found!
```

Diagnosing the problem: With the prescribed grid $q \in [1.02\beta,\ 0.999]$, the capital demand curve $K(q)$ lies strictly above the asset supply curve $E_a(q)$ for all grid points — the curves never cross, so no equilibrium is found.

The issue is that the lower bound $1.02\beta \approx 0.9792$ is too far from β. As $q \to \beta^+$, the interest rate approaches $1/\beta - 1$, which makes precautionary savings very large ($E_a(q) \to \infty$). Meanwhile, $K(q)$ remains finite. The crossing must therefore occur at some q between β and 1.02β, which our grid misses entirely.

Fix: We bring the lower bound closer to β (using 1.002β instead of 1.02β) and increase the asset grid upper bound to $a_{\max} = 200$ to accommodate the larger wealth accumulation near β. We also increase the asset grid to $N_a = 500$ for accuracy.

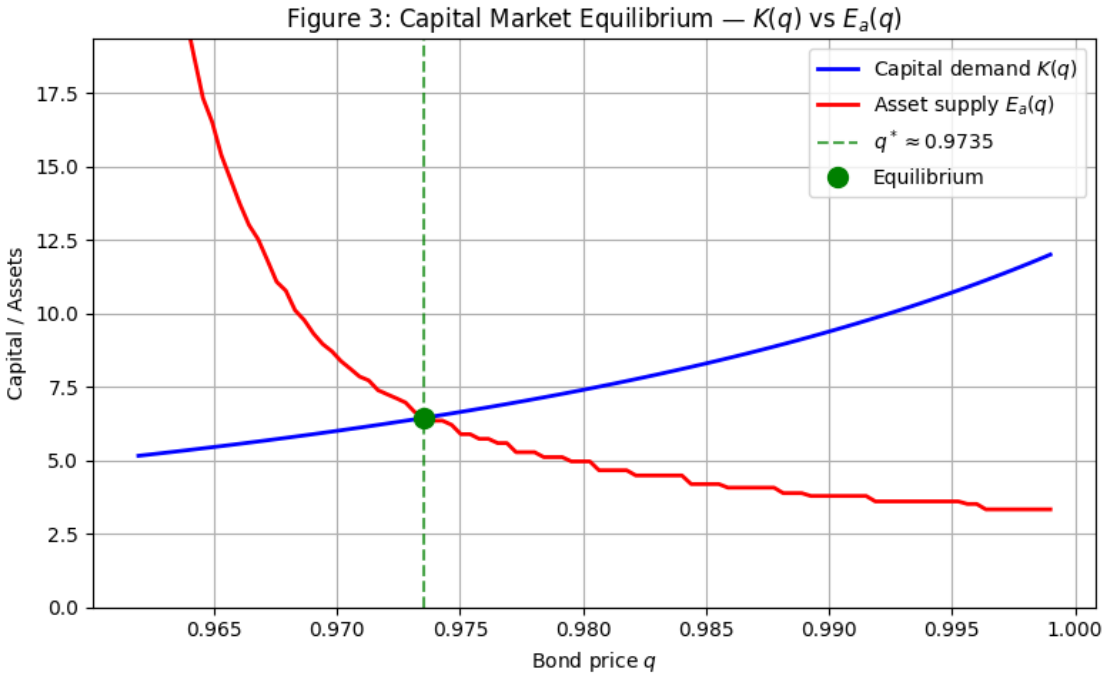
[35]:
```python
# Q3(a)(iii) - Corrected computation with finer grid near beta
N_a = 500
a_max = 200.0
a_grid = np.linspace(0, a_max, N_a)

N_q = 100
q_grid = np.linspace(1.002 * beta, 0.999, N_q)

print("Re-solving with corrected grid...")
print(f"  q range: [{q_grid[0]:.6f}, {q_grid[-1]:.6f}]  (beta = {beta})")
print(f"  a_max = {a_max}, N_a = {N_a}")

V_all = []
g_all = []
g_idx_all = []
dist_all = []
Ea_all = np.zeros(N_q)
K_demand = np.zeros(N_q)

for iq, q_val in enumerate(q_grid):
    V, g, g_idx = solve_vfi(q_val)
    V_all.append(V)
    g_all.append(g)
    g_idx_all.append(g_idx)

    dist = compute_stationary_dist(g_idx)
    dist_all.append(dist)
    Ea_all[iq] = np.sum(a_grid[:, None] * dist)
    K_demand[iq] = K_from_q(q_val)
```

```
    if (iq + 1) % 20 == 0:
        print(f"  [{iq+1:3d}/{N_q}] q={q_val:.4f}  K(q)={K_demand[iq]:7.2f}  Ea(q)={Ea_all[iq]:7.2f}")

# Find equilibrium
eq_idx = np.argmin(np.abs(K_demand - Ea_all))
q_star = q_grid[eq_idx]

print(f"\nEquilibrium found: q*  {q_star:.4f}")
print(f"  K(q*)  = {K_demand[eq_idx]:.4f}")
print(f"  Ea(q*) = {Ea_all[eq_idx]:.4f}")
print(f"  |K - Ea| = {abs(K_demand[eq_idx] - Ea_all[eq_idx]):.4f}")
print(f"  r* = {r_from_q(q_star):.4f},  w* = {w_from_q(q_star):.4f}")

frac_at_top = np.sum(dist_all[eq_idx][-1, :])
print(f"  Fraction at a_max = {frac_at_top:.6f} (should be  0)")
```

```
Re-solving with corrected grid…
  q range: [0.961920, 0.999000]  (beta = 0.96)
  a_max = 200.0, N_a = 500
  [ 20/100] q=0.9690  K(q)=   5.90  Ea(q)=   9.32
  [ 40/100] q=0.9765  K(q)=   6.87  Ea(q)=   5.59
  [ 60/100] q=0.9840  K(q)=   8.11  Ea(q)=   4.49
  [ 80/100] q=0.9915  K(q)=   9.76  Ea(q)=   3.79
  [100/100] q=0.9990  K(q)=  12.01  Ea(q)=   3.33

Equilibrium found: q*  0.9735
  K(q*)  = 6.4501
  Ea(q*) = 6.5063
  |K - Ea| = 0.0562
  r* = 0.0772,  w* = 1.3854
  Fraction at a_max = 0.000000 (should be  0)
```

[37]:
```
# Figure 3 (Corrected): K(q) vs E_a(q)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(q_grid, K_demand, 'b-', linewidth=2, label=r'Capital demand $K(q)$')
ax.plot(q_grid, Ea_all, 'r-', linewidth=2, label=r'Asset supply $E_a(q)$')
ax.axvline(q_star, color='green', linestyle='--', alpha=0.7, label=f'$q^*\\approx {q_star:.4f}$')
ax.plot(q_star, K_demand[eq_idx], 'go', markersize=10, zorder=5,label='Equilibrium')

# Zoom to reasonable range so the crossing is visible
y_upper = min(max(K_demand.max(), Ea_all.max()), 3 * K_demand[eq_idx])
ax.set_ylim(0, y_upper)
ax.set_xlabel('Bond price $q$')
ax.set_ylabel('Capital / Assets')
```

```
ax.set_title('Figure 3: Capital Market Equilibrium - $K(q)$ vs $E_a(q)$')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.savefig('fig3_equilibrium.png', dpi=150)
plt.show()

print(f"Equilibrium: q*  {q_star:.4f}, K(q*) = {K_demand[eq_idx]:.4f}, Ea(q*)␣
  ↪= {Ea_all[eq_idx]:.4f}")
```




```
Equilibrium: q*  0.9735, K(q*) = 6.4501, Ea(q*) = 6.5063
```

[ ]:
```
# Figure 1: Value functions at corrected equilibrium q*
V_eq = V_all[eq_idx]

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(a_grid, V_eq[:, 0], label=f'$s_L = {s_L}$')
ax.plot(a_grid, V_eq[:, 1], label=f'$s_H = {s_H}$')
ax.set_xlabel('Assets $a$')
ax.set_ylabel('$V(a, s)$')
ax.set_title(f'Figure 1: Value Functions at $q^* = {q_star:.4f}$')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.savefig('fig1_value_functions.png', dpi=150)
```

```
plt.show()
```

[ ]:
```
# Figure 2: Policy functions at corrected equilibrium q*
g_eq = g_all[eq_idx]

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(a_grid, g_eq[:, 0], label=f'$s_L = {s_L}$')
ax.plot(a_grid, g_eq[:, 1], label=f'$s_H = {s_H}$')
ax.plot(a_grid, a_grid, 'k--', alpha=0.3, label='45° line')
ax.set_xlabel('Current Assets $a$')
ax.set_ylabel("Next Period Assets $a' = g(a, s)$")
ax.set_title(f'Figure 2: Policy Functions at $q^* = {q_star:.4f}$')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.savefig('fig2_policy_functions.png', dpi=150)
plt.show()
```

1.0.4 Question 4

Once you have sorted out the equilibrium q:

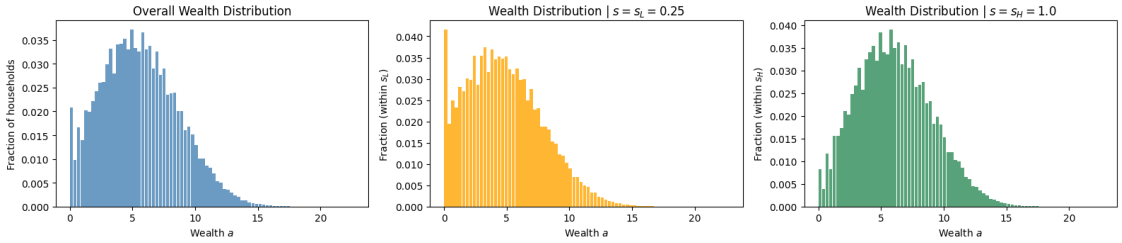
Q4(a) Provide a graph of the distribution with the level of wealth in the horizontal axis and the fraction of households in the vertical axis. Provide similar graphs for the distribution of wealth conditional on the skill levels $s \in \{s_L, s_H\}$. Likewise, produce graphs of the consumption distributions.

[41]:
```
# Q4(a): Wealth distribution (overall and conditional on s)
# Re-solve at equilibrium q* with a finer grid focused on the relevant wealth␣
 ↪range
N_a_fine = 1000
a_max_fine = 30.0
a_grid_fine = np.linspace(0, a_max_fine, N_a_fine)

# Temporarily swap grids
a_grid_old, N_a_old = a_grid, N_a
a_grid, N_a = a_grid_fine, N_a_fine

V_eq_fine, g_eq_fine, g_idx_eq_fine = solve_vfi(q_star)
dist_eq_fine = compute_stationary_dist(g_idx_eq_fine)

# Restore original grid
a_grid, N_a = a_grid_old, N_a_old

# Marginal wealth distributions
dist_total = dist_eq_fine[:, 0] + dist_eq_fine[:, 1]
```

```
dist_sL = dist_eq_fine[:, 0]
dist_sH = dist_eq_fine[:, 1]

# Bin into wider buckets for smooth display
N_w_bins = 80
a_upper = a_grid_fine[np.max(np.where(dist_total > 1e-8))] * 1.1
w_bins = np.linspace(0, a_upper, N_w_bins + 1)
w_centers = 0.5 * (w_bins[:-1] + w_bins[1:])
bin_width = w_bins[1] - w_bins[0]

def bin_distribution(dist_vec, grid, bins):
    binned = np.zeros(len(bins) - 1)
    idx = np.digitize(grid, bins) - 1
    idx = np.clip(idx, 0, len(bins) - 2)
    for i in range(len(grid)):
        binned[idx[i]] += dist_vec[i]
    return binned

dist_total_binned = bin_distribution(dist_total, a_grid_fine, w_bins)
dist_sL_binned = bin_distribution(dist_sL, a_grid_fine, w_bins)
dist_sH_binned = bin_distribution(dist_sH, a_grid_fine, w_bins)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].bar(w_centers, dist_total_binned, width=bin_width*0.9,␣
 ↪color='steelblue', alpha=0.8)
axes[0].set_xlabel('Wealth $a$')
axes[0].set_ylabel('Fraction of households')
axes[0].set_title('Overall Wealth Distribution')

axes[1].bar(w_centers, dist_sL_binned / dist_sL_binned.sum(), width=bin_width*0.
 ↪9, color='orange', alpha=0.8)
axes[1].set_xlabel('Wealth $a$')
axes[1].set_ylabel('Fraction (within $s_L$)')
axes[1].set_title(f'Wealth Distribution | $s = s_L = {s_L}$')

axes[2].bar(w_centers, dist_sH_binned / dist_sH_binned.sum(), width=bin_width*0.
 ↪9, color='seagreen', alpha=0.8)
axes[2].set_xlabel('Wealth $a$')
axes[2].set_ylabel('Fraction (within $s_H$)')
axes[2].set_title(f'Wealth Distribution | $s = s_H = {s_H}$')

plt.suptitle(f'Figure 4: Wealth Distributions at $q^* = {q_star:.4f}$', y=1.02,␣
 ↪fontsize=13)
plt.tight_layout()
plt.savefig('fig4_wealth_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
frac_at_top_fine = np.sum(dist_eq_fine[-1, :])
print(f"Fraction at a_max = {frac_at_top_fine:.6f} (should be  0)")
```

Figure 4: Wealth Distributions at $q^* = 0.9735$

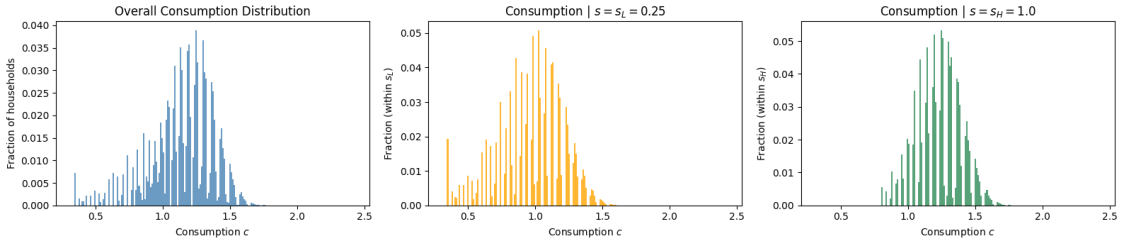



```
Fraction at a_max = 0.000000 (should be  0)
```

[42]:
```
# Q4(a): Consumption distributions (overall and conditional on s)
# Consumption: c(a, s) = w*s + a - q*g(a, s)
w_star = w_from_q(q_star)

# Consumption at each (a, s) pair on the fine grid
c_vals = np.zeros((N_a_fine, 2))
for j in range(2):
    c_vals[:, j] = w_star * s_vals[j] + a_grid_fine - q_star * g_eq_fine[:, j]

# Build consumption distribution by binning
N_c_bins = 200
c_min = c_vals[c_vals > 0].min() * 0.9
c_max = c_vals.max() * 1.1
c_bins = np.linspace(c_min, c_max, N_c_bins + 1)
c_centers = 0.5 * (c_bins[:-1] + c_bins[1:])

dist_c_total = np.zeros(N_c_bins)
dist_c_sL = np.zeros(N_c_bins)
dist_c_sH = np.zeros(N_c_bins)

for j in range(2):
    bin_idx = np.digitize(c_vals[:, j], c_bins) - 1
    bin_idx = np.clip(bin_idx, 0, N_c_bins - 1)
    for i in range(N_a_fine):
        dist_c_total[bin_idx[i]] += dist_eq_fine[i, j]
        if j == 0:
            dist_c_sL[bin_idx[i]] += dist_eq_fine[i, j]
        else:
            dist_c_sH[bin_idx[i]] += dist_eq_fine[i, j]
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].bar(c_centers, dist_c_total, width=c_bins[1]-c_bins[0],␣
 ↪color='steelblue', alpha=0.8)
axes[0].set_xlabel('Consumption $c$')
axes[0].set_ylabel('Fraction of households')
axes[0].set_title('Overall Consumption Distribution')

axes[1].bar(c_centers, dist_c_sL / dist_c_sL.sum(), width=c_bins[1]-c_bins[0],␣
 ↪color='orange', alpha=0.8)
axes[1].set_xlabel('Consumption $c$')
axes[1].set_ylabel('Fraction (within $s_L$)')
axes[1].set_title(f'Consumption | $s = s_L = {s_L}$')

axes[2].bar(c_centers, dist_c_sH / dist_c_sH.sum(), width=c_bins[1]-c_bins[0],␣
 ↪color='seagreen', alpha=0.8)
axes[2].set_xlabel('Consumption $c$')
axes[2].set_ylabel('Fraction (within $s_H$)')
axes[2].set_title(f'Consumption | $s = s_H = {s_H}$')

plt.suptitle(f'Figure 5: Consumption Distributions at $q^* = {q_star:.4f}$',␣
 ↪y=1.02, fontsize=13)
plt.tight_layout()
plt.savefig('fig5_consumption_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```

Figure 5: Consumption Distributions at $q^* = 0.9735$



Q4(b) Compute the basic statistics for the income and wealth distribution of your model, in particular, mean, std., skewness of wealth.

[43]:
```
# Q4(b): Mean, std, skewness of income and wealth (using fine grid)

# --- Wealth statistics ---
Ea = np.sum(a_grid_fine[:, None] * dist_eq_fine)
Ea2 = np.sum(a_grid_fine[:, None]**2 * dist_eq_fine)
Ea3 = np.sum(a_grid_fine[:, None]**3 * dist_eq_fine)
```

```
mean_wealth = Ea
var_wealth = Ea2 - Ea**2
std_wealth = np.sqrt(var_wealth)
skew_wealth = (Ea3 - 3*Ea*var_wealth - Ea**3) / (std_wealth**3)

print("=== Wealth Distribution Statistics ===")
print(f"Mean     = {mean_wealth:.4f}")
print(f"Std      = {std_wealth:.4f}")
print(f"Skewness = {skew_wealth:.4f}")

# --- Income statistics ---
r_star = r_from_q(q_star)
income = np.zeros((N_a_fine, 2))
for j in range(2):
    income[:, j] = w_star * s_vals[j] + (r_star - delta) * a_grid_fine

Ey = np.sum(income * dist_eq_fine)
Ey2 = np.sum(income**2 * dist_eq_fine)
Ey3 = np.sum(income**3 * dist_eq_fine)

mean_income = Ey
var_income = Ey2 - Ey**2
std_income = np.sqrt(var_income)
skew_income = (Ey3 - 3*Ey*var_income - Ey**3) / (std_income**3)

print("\n=== Income Distribution Statistics ===")
print(f"Mean     = {mean_income:.4f}")
print(f"Std      = {std_income:.4f}")
print(f"Skewness = {skew_income:.4f}")

# --- Consumption statistics ---
Ec = np.sum(c_vals * dist_eq_fine)
Ec2 = np.sum(c_vals**2 * dist_eq_fine)
Ec3 = np.sum(c_vals**3 * dist_eq_fine)

mean_cons = Ec
var_cons = Ec2 - Ec**2
std_cons = np.sqrt(var_cons)
skew_cons = (Ec3 - 3*Ec*var_cons - Ec**3) / (std_cons**3)

print("\n=== Consumption Distribution Statistics ===")
print(f"Mean     = {mean_cons:.4f}")
print(f"Std      = {std_cons:.4f}")
print(f"Skewness = {skew_cons:.4f}")

# Summary table
```

```
print("\n=== Summary Table ===")
print(f"{'':15s} {'Mean':>10s} {'Std':>10s} {'Skewness':>10s}")
print("-" * 47)
print(f"{'Wealth':15s} {mean_wealth:10.4f} {std_wealth:10.4f} {skew_wealth:10.
  ↪4f}")
print(f"{'Income':15s} {mean_income:10.4f} {std_income:10.4f} {skew_income:10.
  ↪4f}")
print(f"{'Consumption':15s} {mean_cons:10.4f} {std_cons:10.4f} {skew_cons:10.
  ↪4f}")
```

```
=== Wealth Distribution Statistics ===
Mean     = 5.5932
Std      = 3.0411
Skewness = 0.3630

=== Income Distribution Statistics ===
Mean     = 1.1478
Std      = 0.5258
Skewness = -0.4968

=== Consumption Distribution Statistics ===
Mean     = 1.1438
Std      = 0.2330
Skewness = -0.7810

=== Summary Table ===
                      Mean        Std   Skewness
-----------------------------------------------
Wealth              5.5932     3.0411     0.3630
Income              1.1478     0.5258    -0.4968
Consumption         1.1438     0.2330    -0.7810
```

1.1 Part 2: Aiyagari with Two Types of Households (40 pts)

Imagine now that the economy is very similar to the one above, except that there are two types of households (workers.) Here, 70% of workers are Type I, which is exactly the same as the one above. The rest, 30%, are Type II of workers that are more prone to get sick. The skill transition probabilities for these workers are:

$$Pr[s_{t+1} = s_H] = \begin{cases} 0.5 & \text{if } s_t = s_H \\ 0.5 & \text{if } s_t = s_L \end{cases}$$

$$Pr[s_{t+1} = s_L] = \begin{cases} 0.5 & \text{if } s_t = s_H \\ 0.5 & \text{if } s_t = s_L \end{cases}$$

Extend the code you produced to answer the previous question, explaining carefully how you had to change steps 1–4. Explain, why this form of ex-ante heterogeneity will change the general equilibrium values for the aggregates $\{L, K, Y\}$, the equilibrium value q, and the wealth distribution. Likewise, extend your discussion to not only compare within-types of inequality, but also between-types differences.

Discussion of changes to steps 1–4:

Step 1 (Invariant distribution): Each type has its own transition matrix and hence its own invariant distribution. Type I retains Π_I with $\lambda_I = (3/8, 5/8)$. Type II has $\Pi_{II} = \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix}$, whose invariant distribution is $\lambda_{II} = (1/2, 1/2)$ — Type II workers spend equal time in each skill state.

Step 2 (Aggregate labor and prices): Aggregate labor is now a population-weighted average:

$$\bar{L} = 0.7 \cdot \bar{L}_I + 0.3 \cdot \bar{L}_{II} = 0.7 \sum_s s\, \lambda_I(s) + 0.3 \sum_s s\, \lambda_{II}(s)$$

Since $\bar{L}_{II} = 0.5 \cdot 0.25 + 0.5 \cdot 1.0 = 0.625 < \bar{L}_I = 0.71875$, the new $\bar{L}$ is lower. The functions $K(q)$, $w(q)$, $r(q)$ use the same formulas but with the new $\bar{L}$.

Step 3 (VFI and equilibrium): For each candidate q, we must solve *two* separate household problems — one for Type I (using Π_I) and one for Type II (using Π_{II}). Each type has its own value function, policy function, and stationary distribution. Aggregate asset supply is the weighted average:

$$E_a(q) = 0.7 \cdot E_a^I(q) + 0.3 \cdot E_a^{II}(q)$$

Equilibrium is where $K(q) = E_a(q)$.

Step 4 (Statistics): The overall wealth distribution is the population-weighted mixture of the two type-specific distributions. Within-type statistics are computed from each type's distribution separately; between-type differences compare mean wealth, consumption, etc. across types.

[13]:
```
# Part 2: Type II transition matrix and invariant distribution
Pi_II = np.array([
    [0.5, 0.5],   # from s_L
    [0.5, 0.5]    # from s_H
])
```

[44]:
```
# Part 2: Solve two-type economy

# --- Type II invariant distribution ---
A_mat_II = Pi_II.T - np.eye(2)
A_mat_II[-1, :] = 1.0
b_vec_II = np.array([0.0, 1.0])
lam_II = np.linalg.solve(A_mat_II, b_vec_II)
print(f"Type I  invariant dist:  _I  = ({lam[0]:.4f}, {lam[1]:.4f})")
print(f"Type II invariant dist:  _II = ({lam_II[0]:.4f}, {lam_II[1]:.4f})")

# --- New aggregate labor ---
omega_I, omega_II = 0.7, 0.3
```

```
L_bar_I = np.dot(s_vals, lam)
L_bar_II = np.dot(s_vals, lam_II)
L_bar_2 = omega_I * L_bar_I + omega_II * L_bar_II
print(f"\nL_bar_I  = {L_bar_I:.4f}")
print(f"L_bar_II = {L_bar_II:.4f}")
print(f"L_bar (two-type) = {L_bar_2:.4f}  (vs Part 1: {L_bar:.4f})")

# --- Redefine price functions with new L_bar ---
def r_from_q_2(q):
    return 1/q - 1 + delta

def K_from_q_2(q):
    r = r_from_q_2(q)
    return ((alpha * A / r) ** (1/(1-alpha))) * L_bar_2

def w_from_q_2(q):
    K = K_from_q_2(q)
    return (1-alpha) * A * (K / L_bar_2)**alpha

# --- VFI and stationary dist that accept Pi as argument ---
def solve_vfi_2(q, Pi_type, N_a_loc, a_grid_loc, tol=1e-6, max_iter=2000):
    w = w_from_q_2(q)
    u_mat = []
    for j in range(2):
        c = (w * s_vals[j] + a_grid_loc)[:, None] - q * a_grid_loc[None, :]
        uj = np.full_like(c, -np.inf)
        pos = c > 0
        uj[pos] = u_crra(c[pos])
        u_mat.append(uj)
    V = np.zeros((N_a_loc, 2))
    g_idx = np.zeros((N_a_loc, 2), dtype=int)
    for it in range(max_iter):
        V_new = np.zeros_like(V)
        for j in range(2):
            EV = V @ Pi_type[j]
            val = u_mat[j] + beta * EV[np.newaxis, :]
            g_idx[:, j] = np.argmax(val, axis=1)
            V_new[:, j] = np.max(val, axis=1)
        if np.max(np.abs(V_new - V)) < tol:
            break
        V = V_new.copy()
    return V, a_grid_loc[g_idx], g_idx

def compute_stationary_dist_2(g_idx, Pi_type, N_a_loc, tol=1e-10,␣
 ↪max_iter=10000):
    dist = np.ones((N_a_loc, 2)) / (N_a_loc * 2)
    for it in range(max_iter):
```

```
        dist_new = np.zeros_like(dist)
        for j in range(2):
            for jp in range(2):
                np.add.at(dist_new[:, jp], g_idx[:, j], Pi_type[j, jp] * dist[:
 ↪, j])
        if np.max(np.abs(dist_new - dist)) < tol:
            break
        dist = dist_new
    return dist

# --- Solve for equilibrium ---
N_a_2 = 500
a_max_2 = 200.0
a_grid_2 = np.linspace(0, a_max_2, N_a_2)
N_q_2 = 100
q_grid_2 = np.linspace(1.002 * beta, 0.999, N_q_2)

print("\nSolving two-type economy...")
Ea_I_all = np.zeros(N_q_2)
Ea_II_all = np.zeros(N_q_2)
Ea_2_all = np.zeros(N_q_2)
K_demand_2 = np.zeros(N_q_2)

# Store solutions at each q
V_I_all, g_I_all, gidx_I_all, dist_I_all = [], [], [], []
V_II_all, g_II_all, gidx_II_all, dist_II_all = [], [], [], []

for iq, q_val in enumerate(q_grid_2):
    # Type I
    V_I, g_I, gidx_I = solve_vfi_2(q_val, Pi, N_a_2, a_grid_2)
    dist_I = compute_stationary_dist_2(gidx_I, Pi, N_a_2)
    V_I_all.append(V_I); g_I_all.append(g_I); gidx_I_all.append(gidx_I);␣
 ↪dist_I_all.append(dist_I)
    Ea_I_all[iq] = np.sum(a_grid_2[:, None] * dist_I)

    # Type II
    V_II, g_II, gidx_II = solve_vfi_2(q_val, Pi_II, N_a_2, a_grid_2)
    dist_II = compute_stationary_dist_2(gidx_II, Pi_II, N_a_2)
    V_II_all.append(V_II); g_II_all.append(g_II); gidx_II_all.append(gidx_II);␣
 ↪dist_II_all.append(dist_II)
    Ea_II_all[iq] = np.sum(a_grid_2[:, None] * dist_II)

    # Aggregate
    Ea_2_all[iq] = omega_I * Ea_I_all[iq] + omega_II * Ea_II_all[iq]
    K_demand_2[iq] = K_from_q_2(q_val)

    if (iq + 1) % 20 == 0:
```

```
        print(f"  [{iq+1:3d}/{N_q_2}] q={q_val:.4f}  K={K_demand_2[iq]:7.2f}  Ea={Ea_2_all[iq]:7.2f}  (I:{Ea_I_all[iq]:.2f}  II:{Ea_II_all[iq]:.2f})")

# Find equilibrium
eq_idx_2 = np.argmin(np.abs(K_demand_2 - Ea_2_all))
q_star_2 = q_grid_2[eq_idx_2]

print(f"\nTwo-type equilibrium: q*  {q_star_2:.4f}  (Part 1: {q_star:.4f})")
print(f"  K(q*)  = {K_demand_2[eq_idx_2]:.4f}")
print(f"  Ea(q*) = {Ea_2_all[eq_idx_2]:.4f}")
print(f"  |K - Ea| = {abs(K_demand_2[eq_idx_2] - Ea_2_all[eq_idx_2]):.4f}")
print(f"  r* = {r_from_q_2(q_star_2):.4f},  w* = {w_from_q_2(q_star_2):.4f}")

# Equilibrium aggregates
K_star_2 = K_from_q_2(q_star_2)
Y_star_2 = A * K_star_2**alpha * L_bar_2**(1-alpha)
K_star_1 = K_from_q(q_star)
Y_star_1 = A * K_star_1**alpha * L_bar**(1-alpha)
print(f"\n=== Aggregate Comparison ===")
print(f"{'':12s} {'Part 1':>10s} {'Part 2':>10s}")
print("-" * 34)
print(f"{'L':12s} {L_bar:10.4f} {L_bar_2:10.4f}")
print(f"{'K':12s} {K_star_1:10.4f} {K_star_2:10.4f}")
print(f"{'Y':12s} {Y_star_1:10.4f} {Y_star_2:10.4f}")
print(f"{'q*':12s} {q_star:10.4f} {q_star_2:10.4f}")
print(f"{'r*':12s} {r_from_q(q_star):10.4f} {r_from_q_2(q_star_2):10.4f}")
print(f"{'w*':12s} {w_from_q(q_star):10.4f} {w_from_q_2(q_star_2):10.4f}")
```

```
Type I  invariant dist:  _I  = (0.3750, 0.6250)
Type II invariant dist:  _II = (0.5000, 0.5000)

L_bar_I  = 0.7188
L_bar_II = 0.6250
L_bar (two-type) = 0.6906  (vs Part 1: 0.7188)

Solving two-type economy…
  [ 20/100] q=0.9690  K=   5.67  Ea=   7.63  (I:9.32  II:3.68)
  [ 40/100] q=0.9765  K=   6.60  Ea=   4.55  (I:5.59  II:2.13)
  [ 60/100] q=0.9840  K=   7.80  Ea=   3.72  (I:4.49  II:1.93)
  [ 80/100] q=0.9915  K=   9.38  Ea=   3.18  (I:3.79  II:1.74)
  [100/100] q=0.9990  K=  11.54  Ea=   2.86  (I:3.33  II:1.76)

Two-type equilibrium: q*  0.9720  (Part 1: 0.9735)
  K(q*)  = 6.0118
  Ea(q*) = 5.9434
  |K - Ea| = 0.0684
  r* = 0.0788,  w* = 1.3714
```

```
=== Aggregate Comparison ===
                  Part 1     Part 2
----------------------------------
L                 0.7188     0.6906
K                 6.4501     6.0118
Y                 1.4936     1.4207
q*                0.9735     0.9720
r*                0.0772     0.0788
w*                1.3854     1.3714
```

[45]:
```
# Part 2: Graphs - wealth and consumption distributions by type

# Re-solve at equilibrium with finer grid for plotting
N_a_2f = 1000
a_max_2f = 30.0
a_grid_2f = np.linspace(0, a_max_2f, N_a_2f)

V_I_fine, g_I_fine, gidx_I_fine = solve_vfi_2(q_star_2, Pi, N_a_2f, a_grid_2f)
dist_I_fine = compute_stationary_dist_2(gidx_I_fine, Pi, N_a_2f)

V_II_fine, g_II_fine, gidx_II_fine = solve_vfi_2(q_star_2, Pi_II, N_a_2f,␣
 ↪a_grid_2f)
dist_II_fine = compute_stationary_dist_2(gidx_II_fine, Pi_II, N_a_2f)

# Overall distribution (population-weighted)
dist_overall_2 = omega_I * dist_I_fine + omega_II * dist_II_fine
dist_total_I = dist_I_fine[:, 0] + dist_I_fine[:, 1]
dist_total_II = dist_II_fine[:, 0] + dist_II_fine[:, 1]
dist_total_2 = dist_overall_2[:, 0] + dist_overall_2[:, 1]

# --- Bin for smooth display ---
N_wb = 80
a_up = a_grid_2f[np.max(np.where(dist_total_2 > 1e-8))] * 1.1
wb = np.linspace(0, a_up, N_wb + 1)
wc = 0.5 * (wb[:-1] + wb[1:])
bw = wb[1] - wb[0]

def bin_dist(d, grid, bins):
    out = np.zeros(len(bins)-1)
    idx = np.clip(np.digitize(grid, bins) - 1, 0, len(bins)-2)
    for i in range(len(grid)):
        out[idx[i]] += d[i]
    return out

# Figure 6: Wealth distributions by type
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
```

```
axes[0].bar(wc, bin_dist(dist_total_2, a_grid_2f, wb), width=bw*0.9,␣
 ↪color='steelblue', alpha=0.8)
axes[0].set_xlabel('Wealth $a$'); axes[0].set_ylabel('Fraction of households')
axes[0].set_title('Overall Wealth Distribution')

axes[1].bar(wc, bin_dist(dist_total_I, a_grid_2f, wb), width=bw*0.9,␣
 ↪color='royalblue', alpha=0.8)
axes[1].set_xlabel('Wealth $a$'); axes[1].set_ylabel('Fraction (Type I)')
axes[1].set_title('Type I (70%)')

axes[2].bar(wc, bin_dist(dist_total_II, a_grid_2f, wb), width=bw*0.9,␣
 ↪color='crimson', alpha=0.8)
axes[2].set_xlabel('Wealth $a$'); axes[2].set_ylabel('Fraction (Type II)')
axes[2].set_title('Type II (30%)')

plt.suptitle(f'Figure 6: Wealth Distributions - Two-Type Economy at $q^* =␣
 ↪{q_star_2:.4f}$', y=1.02, fontsize=13)
plt.tight_layout()
plt.savefig('fig6_wealth_two_type.png', dpi=150, bbox_inches='tight')
plt.show()

# --- Consumption distributions ---
w_star_2 = w_from_q_2(q_star_2)
c_I = np.zeros((N_a_2f, 2))
c_II = np.zeros((N_a_2f, 2))
for j in range(2):
    c_I[:, j] = w_star_2 * s_vals[j] + a_grid_2f - q_star_2 * g_I_fine[:, j]
    c_II[:, j] = w_star_2 * s_vals[j] + a_grid_2f - q_star_2 * g_II_fine[:, j]

# Bin consumption
all_c = np.concatenate([c_I.ravel(), c_II.ravel()])
cb = np.linspace(all_c[all_c > 0].min() * 0.9, all_c.max() * 1.1, 81)
cc = 0.5 * (cb[:-1] + cb[1:])
cbw = cb[1] - cb[0]

def bin_c_dist(c_arr, dist_arr, bins):
    out = np.zeros(len(bins)-1)
    for j in range(2):
        idx = np.clip(np.digitize(c_arr[:, j], bins) - 1, 0, len(bins)-2)
        for i in range(len(c_arr)):
            out[idx[i]] += dist_arr[i, j]
    return out

dc_I = bin_c_dist(c_I, dist_I_fine, cb)
dc_II = bin_c_dist(c_II, dist_II_fine, cb)
dc_all = omega_I * dc_I + omega_II * dc_II
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].bar(cc, dc_all, width=cbw*0.9, color='steelblue', alpha=0.8)
axes[0].set_xlabel('Consumption $c$'); axes[0].set_ylabel('Fraction')
axes[0].set_title('Overall Consumption')

axes[1].bar(cc, dc_I, width=cbw*0.9, color='royalblue', alpha=0.8)
axes[1].set_xlabel('Consumption $c$'); axes[1].set_ylabel('Fraction (Type I)')
axes[1].set_title('Type I (70%)')

axes[2].bar(cc, dc_II, width=cbw*0.9, color='crimson', alpha=0.8)
axes[2].set_xlabel('Consumption $c$'); axes[2].set_ylabel('Fraction (Type II)')
axes[2].set_title('Type II (30%)')

plt.suptitle(f'Figure 7: Consumption Distributions - Two-Type Economy at $q^* =␣
 ↪{q_star_2:.4f}$', y=1.02, fontsize=13)
plt.tight_layout()
plt.savefig('fig7_consumption_two_type.png', dpi=150, bbox_inches='tight')
plt.show()
```

Figure 6: Wealth Distributions — Two-Type Economy at $q^* = 0.9720$

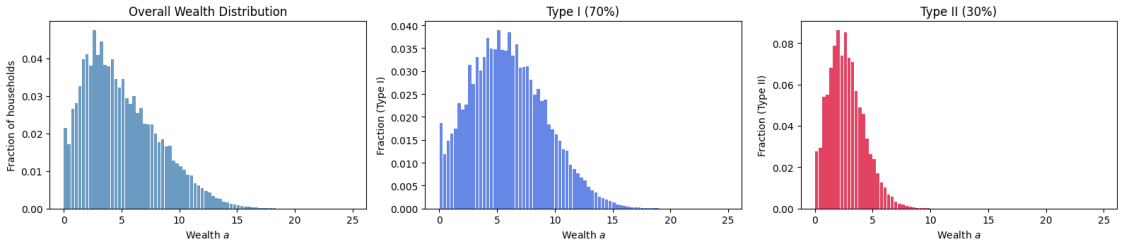
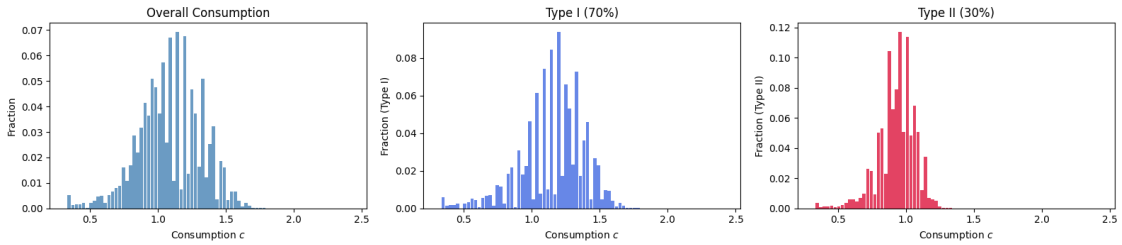


Figure 7: Consumption Distributions — Two-Type Economy at $q^* = 0.9720$



[46]:
```
# Part 2: Statistics - within-type and between-type inequality

def compute_stats(grid, dist_2d):
    """Compute mean, std, skewness from a 2D distribution over (a, s)."""
```

```
    E1 = np.sum(grid[:, None] * dist_2d)
    E2 = np.sum(grid[:, None]**2 * dist_2d)
    E3 = np.sum(grid[:, None]**3 * dist_2d)
    var = E2 - E1**2
    std = np.sqrt(max(var, 0))
    skew = (E3 - 3*E1*var - E1**3) / (std**3) if std > 0 else 0.0
    return E1, std, skew

def compute_c_stats(c_arr, dist_2d):
    """Compute mean, std, skewness of consumption."""
    E1 = np.sum(c_arr * dist_2d)
    E2 = np.sum(c_arr**2 * dist_2d)
    E3 = np.sum(c_arr**3 * dist_2d)
    var = E2 - E1**2
    std = np.sqrt(max(var, 0))
    skew = (E3 - 3*E1*var - E1**3) / (std**3) if std > 0 else 0.0
    return E1, std, skew

# --- Within-type wealth stats ---
mean_w_I, std_w_I, skew_w_I = compute_stats(a_grid_2f, dist_I_fine)
mean_w_II, std_w_II, skew_w_II = compute_stats(a_grid_2f, dist_II_fine)
mean_w_all, std_w_all, skew_w_all = compute_stats(a_grid_2f, dist_overall_2)

print("=== Wealth Statistics ===")
print(f"{'':12s} {'Mean':>10s} {'Std':>10s} {'Skewness':>10s}")
print("-" * 44)
print(f"{'Type I':12s} {mean_w_I:10.4f} {std_w_I:10.4f} {skew_w_I:10.4f}")
print(f"{'Type II':12s} {mean_w_II:10.4f} {std_w_II:10.4f} {skew_w_II:10.4f}")
print(f"{'Overall':12s} {mean_w_all:10.4f} {std_w_all:10.4f} {skew_w_all:10.
 ↪4f}")

# --- Within-type consumption stats ---
mean_c_I, std_c_I, skew_c_I = compute_c_stats(c_I, dist_I_fine)
mean_c_II, std_c_II, skew_c_II = compute_c_stats(c_II, dist_II_fine)
c_overall = omega_I * c_I  # not meaningful to add, use binned approach
# For overall consumption stats, weight the type-specific distributions
dist_c_overall = omega_I * dist_I_fine  # placeholder
c_all_combined = np.concatenate([c_I.ravel(), c_II.ravel()])
d_all_combined = np.concatenate([omega_I * dist_I_fine.ravel(), omega_II *␣
 ↪dist_II_fine.ravel()])
mean_c_all = np.sum(c_all_combined * d_all_combined)
var_c_all = np.sum(c_all_combined**2 * d_all_combined) - mean_c_all**2
std_c_all = np.sqrt(max(var_c_all, 0))
skew_c_all = (np.sum(c_all_combined**3 * d_all_combined) -␣
 ↪3*mean_c_all*var_c_all - mean_c_all**3) / (std_c_all**3) if std_c_all > 0␣
 ↪else 0.0
```

```
print("\n=== Consumption Statistics ===")
print(f"{'':12s} {'Mean':>10s} {'Std':>10s} {'Skewness':>10s}")
print("-" * 44)
print(f"{'Type I':12s} {mean_c_I:10.4f} {std_c_I:10.4f} {skew_c_I:10.4f}")
print(f"{'Type II':12s} {mean_c_II:10.4f} {std_c_II:10.4f} {skew_c_II:10.4f}")
print(f"{'Overall':12s} {mean_c_all:10.4f} {std_c_all:10.4f} {skew_c_all:10.
 ↪4f}")

# --- Income stats ---
r_star_2 = r_from_q_2(q_star_2)
inc_I = np.zeros((N_a_2f, 2))
inc_II = np.zeros((N_a_2f, 2))
for j in range(2):
    inc_I[:, j] = w_star_2 * s_vals[j] + (r_star_2 - delta) * a_grid_2f
    inc_II[:, j] = w_star_2 * s_vals[j] + (r_star_2 - delta) * a_grid_2f

mean_y_I, std_y_I, skew_y_I = compute_c_stats(inc_I, dist_I_fine)
mean_y_II, std_y_II, skew_y_II = compute_c_stats(inc_II, dist_II_fine)

inc_combined = np.concatenate([inc_I.ravel(), inc_II.ravel()])
d_combined = np.concatenate([omega_I * dist_I_fine.ravel(), omega_II *␣
 ↪dist_II_fine.ravel()])
mean_y_all = np.sum(inc_combined * d_combined)
var_y_all = np.sum(inc_combined**2 * d_combined) - mean_y_all**2
std_y_all = np.sqrt(max(var_y_all, 0))
skew_y_all = (np.sum(inc_combined**3 * d_combined) - 3*mean_y_all*var_y_all -␣
 ↪mean_y_all**3) / (std_y_all**3) if std_y_all > 0 else 0.0

print("\n=== Income Statistics ===")
print(f"{'':12s} {'Mean':>10s} {'Std':>10s} {'Skewness':>10s}")
print("-" * 44)
print(f"{'Type I':12s} {mean_y_I:10.4f} {std_y_I:10.4f} {skew_y_I:10.4f}")
print(f"{'Type II':12s} {mean_y_II:10.4f} {std_y_II:10.4f} {skew_y_II:10.4f}")
print(f"{'Overall':12s} {mean_y_all:10.4f} {std_y_all:10.4f} {skew_y_all:10.
 ↪4f}")

# --- Between-type differences ---
print("\n=== Between-Type Differences ===")
print(f"Mean wealth gap (I - II):      {mean_w_I - mean_w_II:+.4f}")
print(f"Mean consumption gap (I - II): {mean_c_I - mean_c_II:+.4f}")
print(f"Mean income gap (I - II):      {mean_y_I - mean_y_II:+.4f}")
print(f"Std wealth - Type I: {std_w_I:.4f}, Type II: {std_w_II:.4f}")
print(f"Std consumption - Type I: {std_c_I:.4f}, Type II: {std_c_II:.4f}")
```

```
=== Wealth Statistics ===
                    Mean        Std    Skewness
--------------------------------------------
```

```
Type I           5.9927     3.2409     0.3960
Type II          2.7441     1.5310     0.6385
Overall          5.0181     3.2050     0.7273

=== Consumption Statistics ===
                   Mean        Std   Skewness
-----------------------------------------------
Type I           1.1533     0.2330    -0.7268
Type II          0.9339     0.1348    -0.8869
Overall          1.0875     0.2314    -0.2406

=== Income Statistics ===
                   Mean        Std   Skewness
-----------------------------------------------
Type I           1.1581     0.5236    -0.4908
Type II          0.9361     0.5162     0.0004
Overall          1.0915     0.5312    -0.3278

=== Between-Type Differences ===
Mean wealth gap (I - II):      +3.2486
Mean consumption gap (I - II): +0.2194
Mean income gap (I - II):      +0.2220
Std wealth - Type I: 3.2409, Type II: 1.5310
Std consumption - Type I: 0.2330, Type II: 0.1348
```

Discussion: Impact of ex-ante heterogeneity on equilibrium and inequality

Aggregates $\{L, K, Y\}$: Type II workers have invariant distribution $\lambda_{II} = (0.5, 0.5)$ versus Type I's $(0.375, 0.625)$. Since Type II spends more time in the low-skill state, $\bar{L}_{II} = 0.625 < \bar{L}_I = 0.71875$. The population-weighted $\bar{L}$ therefore falls. With lower effective labor supply, both K and Y decrease in equilibrium (since $K = (\alpha A/r)^{1/(1-\alpha)}\bar{L}$ scales linearly in $\bar{L}$, and $Y = AK^{\alpha}\bar{L}^{1-\alpha}$).

Equilibrium q: Type II workers face more volatile income (autocorrelation = 0 vs 0.6 for Type I). This greater uninsurable risk increases their precautionary savings motive, pushing up aggregate asset supply $E_a(q)$ for any given q. To restore equilibrium ($K(q) = E_a(q)$), q must adjust — likely rising (lower interest rate) to discourage savings. The net effect depends on the relative magnitude of the $\bar{L}$ decrease (which lowers $K(q)$) and the precautionary savings increase.

Wealth distribution: Type II workers have higher wealth dispersion because their income process is more volatile but has no persistence (i.i.d.). They cannot rely on skill persistence to smooth consumption, so they save more precautionarily. However, their *mean* wealth may differ from Type I's depending on how equilibrium prices adjust. The overall wealth distribution is a mixture of two sub-distributions, which increases overall inequality beyond what either type alone would generate.

Within-type vs. between-type inequality: Within-type inequality is driven by idiosyncratic skill shocks and the borrowing constraint. Type II should exhibit higher within-type wealth dispersion due to the more volatile (lower autocorrelation) income process. Between-type inequality arises from the different ergodic skill distributions: Type I workers earn $\bar{L}_I = 0.719$ per capita while Type II earns $\bar{L}_{II} = 0.625$. This permanent earnings gap creates a between-type component of inequality that cannot be eliminated by any amount of self-insurance. The overall variance of

wealth decomposes as:

$$\text{Var}(a) = \underbrace{E[\text{Var}(a|\text{type})]}_{\text{within}} + \underbrace{\text{Var}(E[a|\text{type}])}_{\text{between}}$$

The introduction of Type II adds a positive between-type component, unambiguously increasing total inequality.